

Custom AI Logic

ЗАНЯТИЕ 14

Инструктор — **Андрей Помелов**

Проблема: AI не управляет логикой

Искусственный интеллект — мощный инструмент, но у него есть принципиальное ограничение: **он не управляет логикой приложения и не контролирует бизнес-правила** самостоятельно.

Что AI умеет

Генерировать текст, отвечать на вопросы, обрабатывать данные

Что AI не умеет

Принимать решения о доступе, контролировать поток приложения, применять бизнес-правила

Зачем нужна ручная логика?

Чтобы система работала предсказуемо, разработчик должен **явно прописывать правила и решения** — AI их не придумает за вас.

Управление поведением

Определяет, что происходит при каждом действии пользователя

Принятие решений

Ветвление логики на основе данных, ролей и контекста

Бизнес-правила

Ограничения доступа, валидация, политики приложения



РОЛЬ AI В СИСТЕМЕ

AI обрабатывает — не управляет

AI выполняет конкретную задачу: **генерирует ответ или обрабатывает текст**. Но он не знает, кто пользователь, какие у него права и что должно произойти дальше.

i Контроль действий пользователя — ответственность разработчика, а не модели.

Связка компонентов системы

Работающее AI-приложение — это всегда **три взаимодействующих уровня**. Каждый отвечает за свою зону ответственности.



API

AI

Ручная логика

Только совместная работа всех трёх уровней даёт управляемую, надёжную систему.

Пример: получение данных через API

Используем **Axios** для запроса данных о продуктах. AI затем обрабатывает полученный ответ — но сам запрос и обработка ошибок написаны вручную.

```
const axios = require('axios');

async function getProducts() {
  try {
    const response = await axios.get(
      'https://api.example.com/products'
    );
    const products = response.data;

    // Передаём данные в AI для обработки
    return products;
  } catch (error) {
    console.error('Ошибка запроса:', error.message);
    return [];
  }
}
```

❏ Логика обработки ошибок и формат запроса — полностью в руках разработчика.

Проверка ролей пользователя

Условия в коде определяют, **что разрешено конкретному пользователю**. AI не имеет доступа к этим данным и не может принять такое решение.

```
function checkAccess(user, action) {  
  if (user.role === 'admin') {  
    return true; // Полный доступ  
  }  
  
  if (user.role === 'editor' && action !== 'delete') {  
    return true; // Редактор не может удалять  
  }  
  
  return false; // Доступ запрещён  
}
```

admin

Полный доступ ко всем действиям

editor

Чтение и редактирование, без удаления

guest

Доступ запрещён по умолчанию

Имитация базы данных

На раннем этапе разработки можно **хранить данные в файле** (JSON или JS-объект) и работать с ними через обычные функции — без реальной БД.

```
// data.js
const users = [
  { id: 1, name: "Анна", role: "admin" },
  { id: 2, name: "Борис", role: "editor" },
  { id: 3, name: "Вера", role: "guest" }
];

function getUserById(id) {
  return users.find(u => u.id === id);
}

module.exports = { getUserById };
```

- ✓ Такой подход позволяет быстро прототипировать логику без настройки полноценной базы данных.

Плюсы

Быстро, просто, нет зависимостей

Ограничения

Данные не сохраняются между запусками, не масштабируется



Итог: AI — инструмент, логика — в вашем коде

1

AI обрабатывает текст и данные

Генерация, классификация, анализ — зона ответственности модели

2

Логика управления — в коде

Роли, доступ, бизнес-правила — всё прописывается разработчиком вручную

3

Три компонента работают вместе

API + AI + ручная логика = управляемая и надёжная система